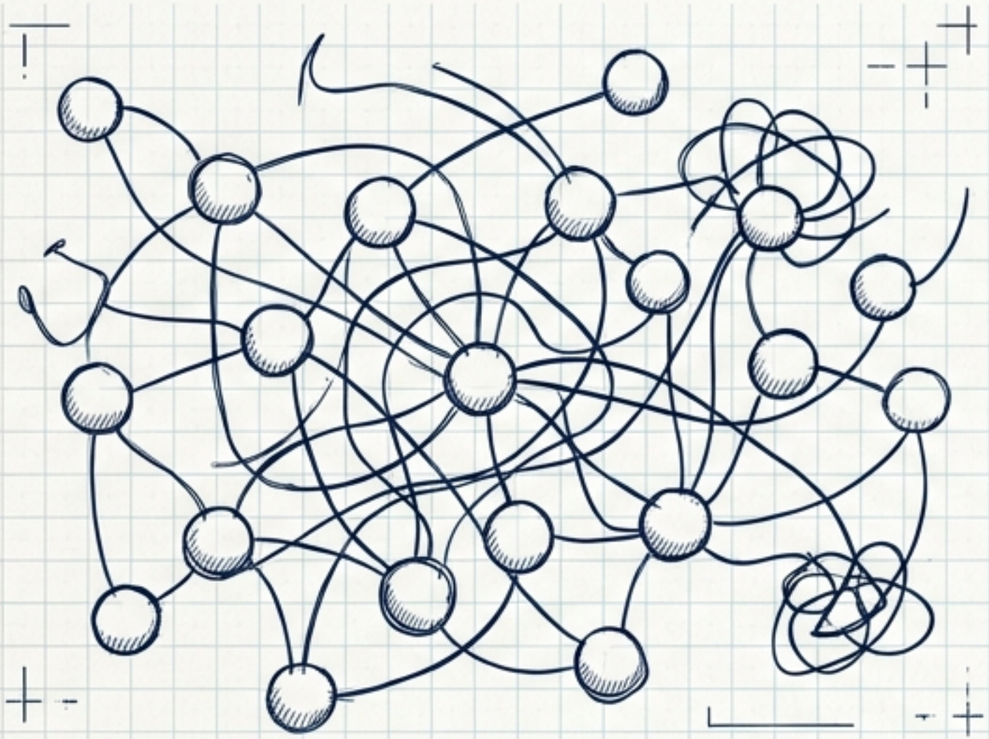
The Explorer-to-Villager Methodology

Raising the Quality Floor for Version 0.13.30+

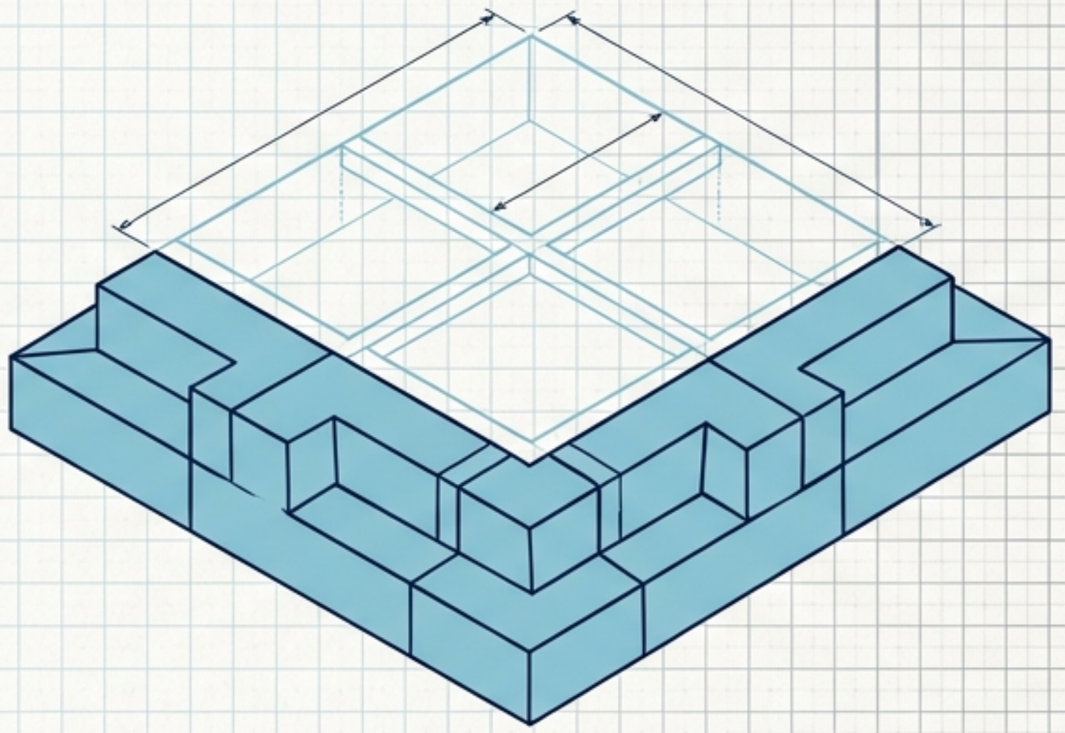
Type:	Process Brief
From:	Project Lead
To:	Dev / Arch / QA / Sec / Design

Projects are reaching structural maturity.

The Present: Rapid Validation



The Imperative: Structural Foundation

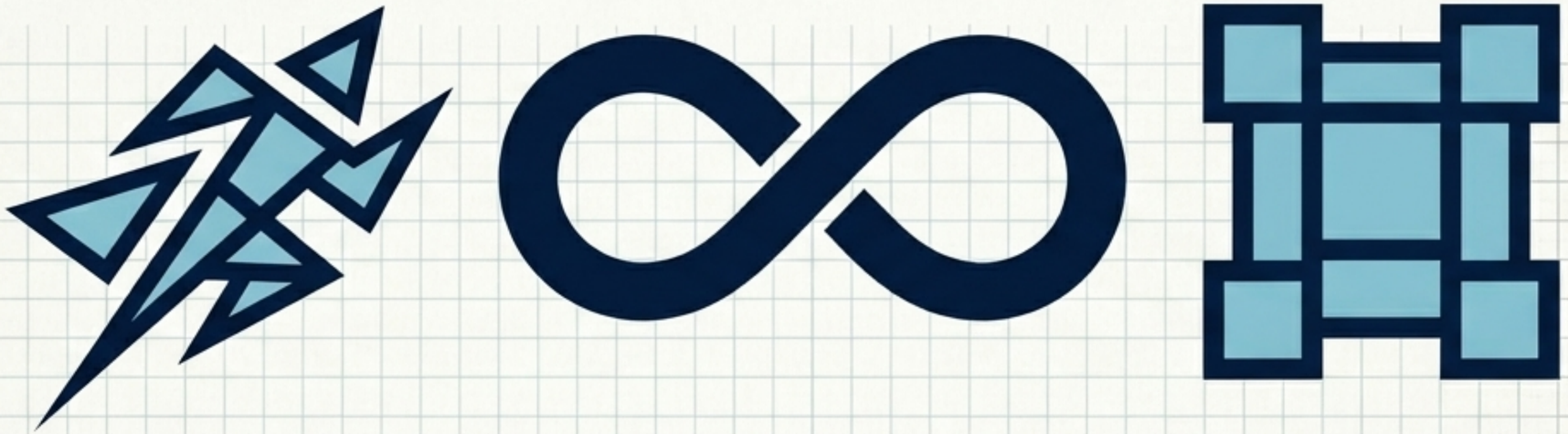


The Context: The CLI project proves we can build fast. Iteration in the flow state works. The features exist.

The Reality: Organic growth accrues friction. The current foundation cannot safely support the weight of the next major release.

The Mandate: Structured quality investment is strictly required before the next wave of major features (unified vault, remotes, branch models) lands.

Two mindsets driving a continuous cycle.



The Explorer

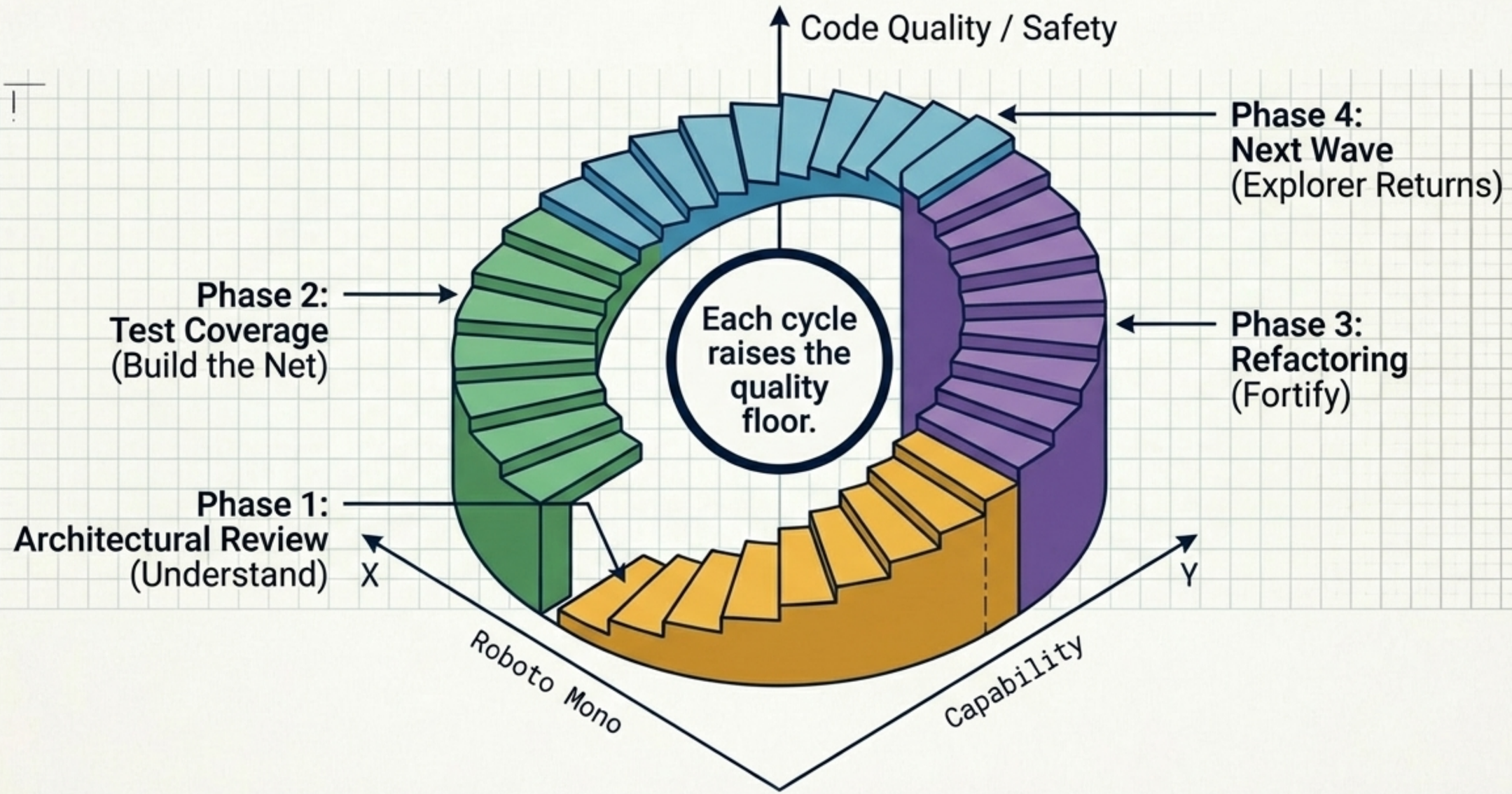
- Builds fast.
- Iterates organically.
- Discovers the product and pushes boundaries.

The Villager

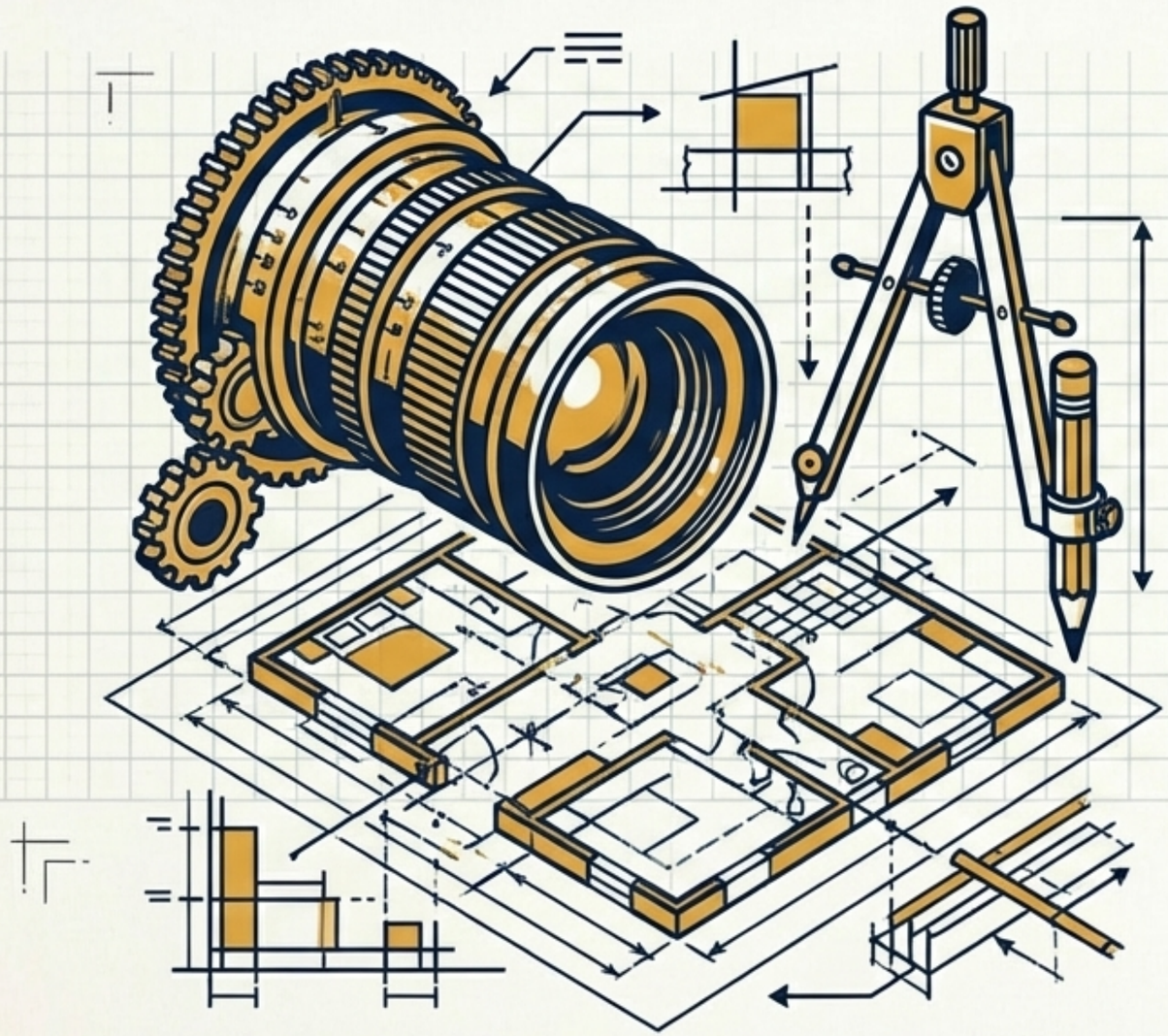
- Consolidates.
- Structures.
- Tests meticulously.
- Fortifies the ground gained.

The Explorer builds. The Villager improves. Then the Explorer builds again on a solid foundation.

The Four-Phase Ascending Cycle



Phase 1: Architectural Review



**RULE: NO FIXING.
ONLY UNDERSTANDING.**

The Objective

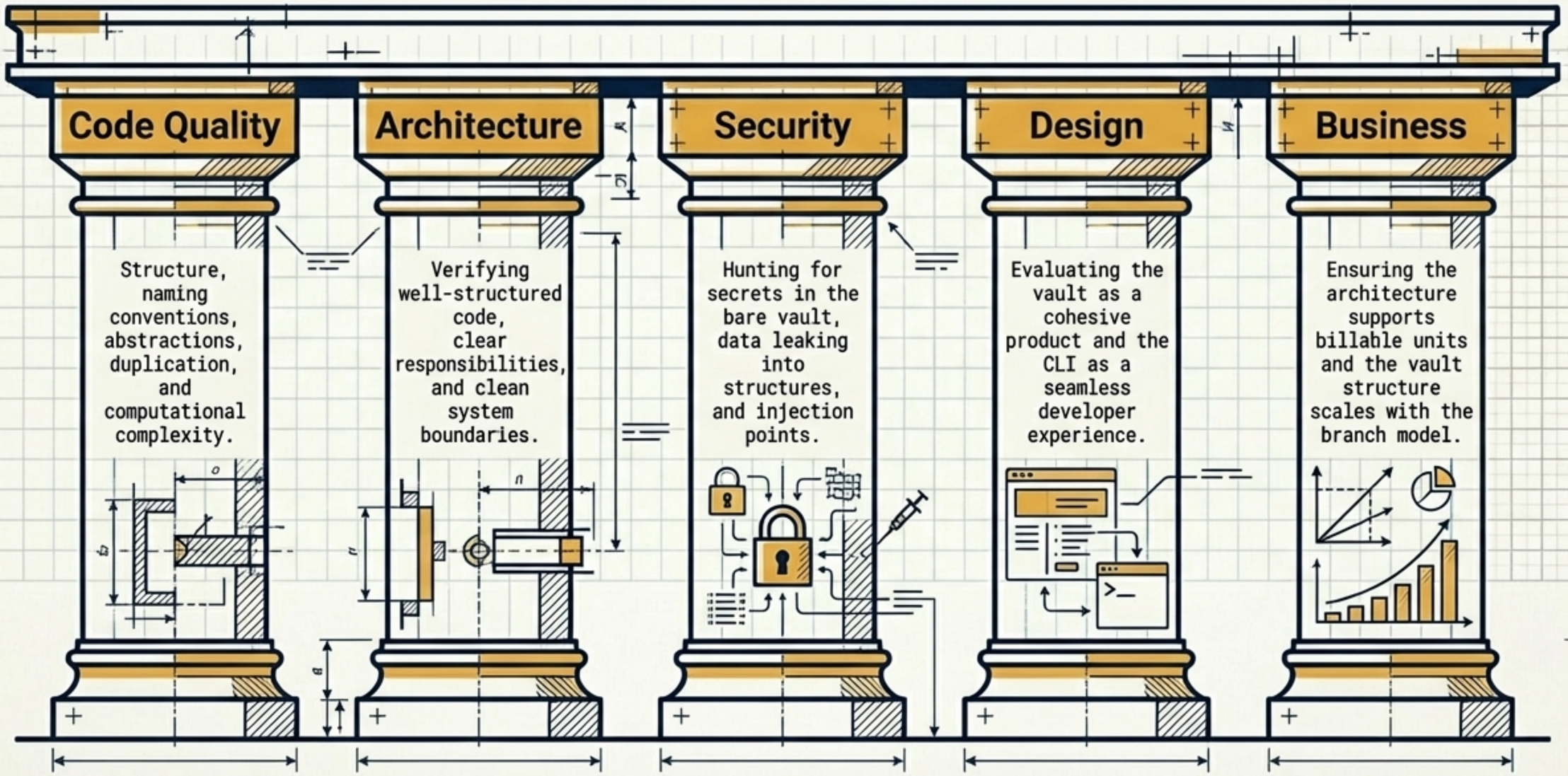
Put fresh eyes on everything that currently exists. Identify structural gaps and map the terrain without altering it.

The Output

A comprehensive findings document. Zero code changes. Zero fixes. Just a clear map of what needs attention.

The 5 Angles of Assessment

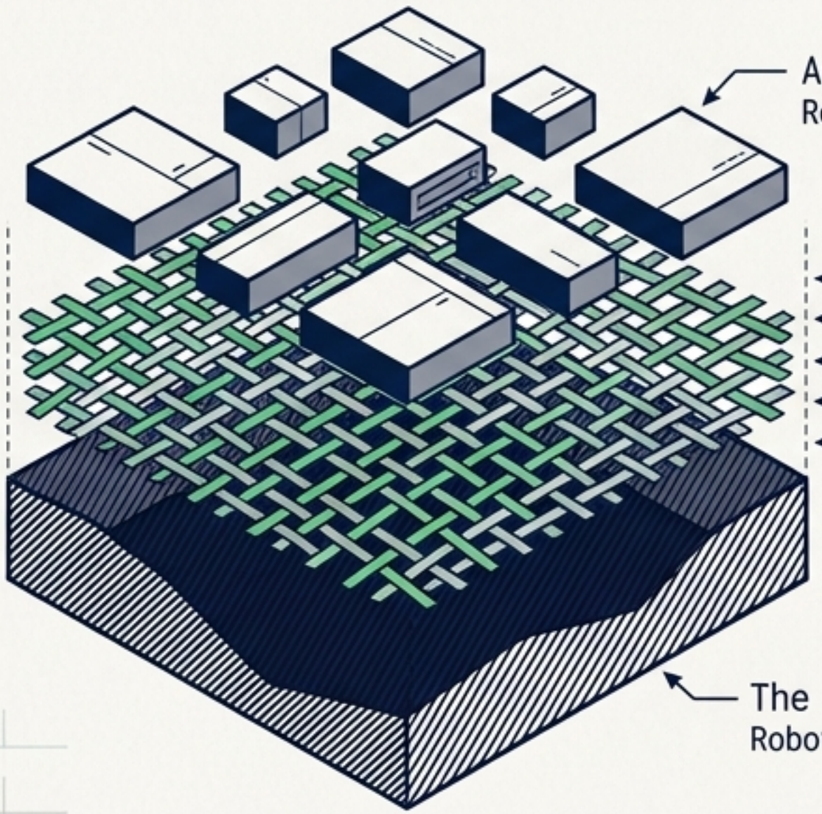
Executed jointly by: Architect, Developer, QA, AppSec, Sherpa, and Designer.



Phase 2: Test Coverage

RULE: NO BEHAVIOUR CHANGES. ONLY TEST CODE WRITTEN.

The Safety Net



The Objective

Build a safety net dense enough to allow fearless refactoring in Phase 3.

The Layers

Unit (functions),
Integration (components),
E2E (full workflows),
CI Pipeline (automated).

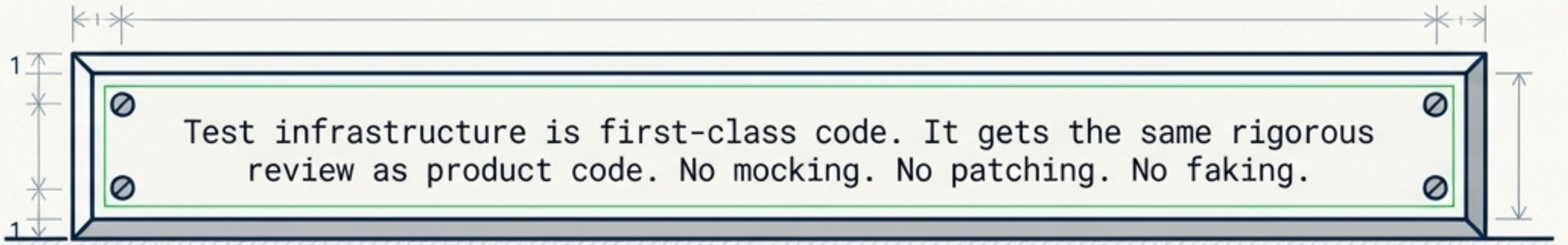
The Priority Metric

Use Case > Line Coverage. Priority is verifying that a user can execute every required journey.

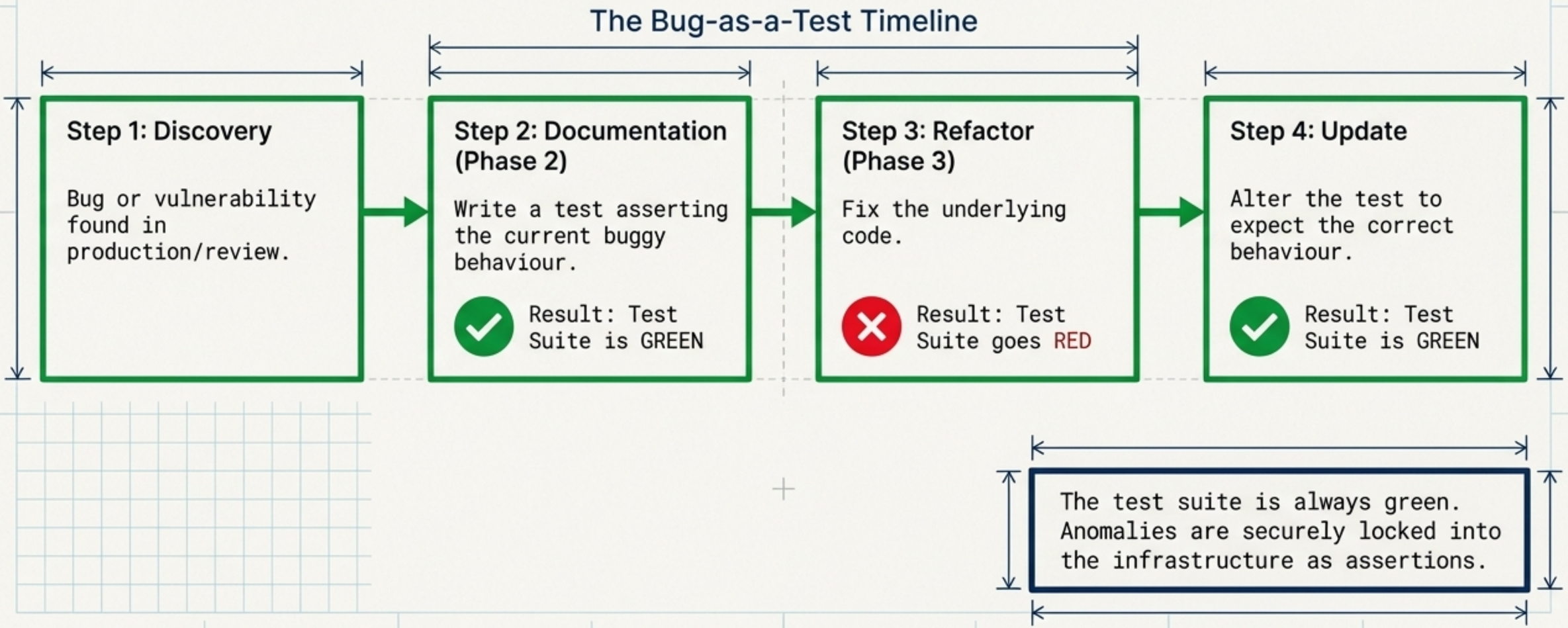
The “No Mocks” Mandate

Standard Practice	Villager Methodology
Need a server? —> Mock the response.	→ Spin up a real SG/end server instance.
Need a vault? → Patch the file system.	→ Create a real vault programmatically.
Need a user flow? → Fake the state transition.	→ Run the real end-to-end flow.

HARD RULE



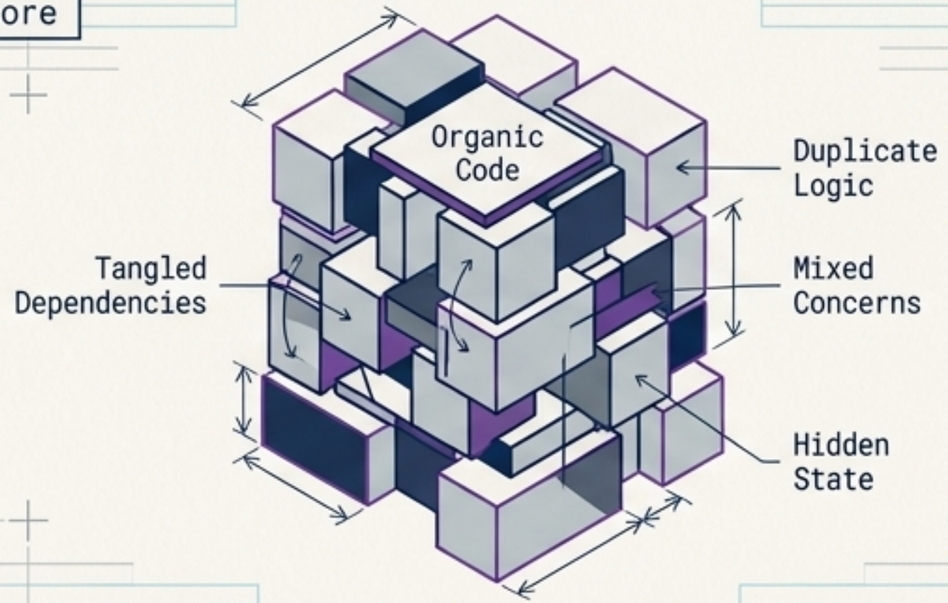
Paradigm Shift: Bugs as Passing Tests



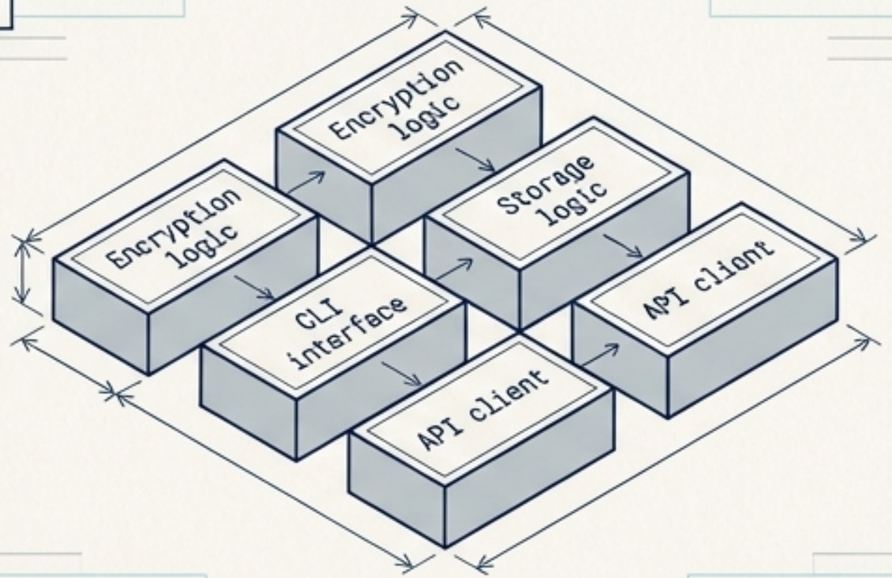
Phase 3: Refactoring

RULE: ALL TESTS PASS AFTER EVERY CHANGE. RED CI = REVERT.

Before

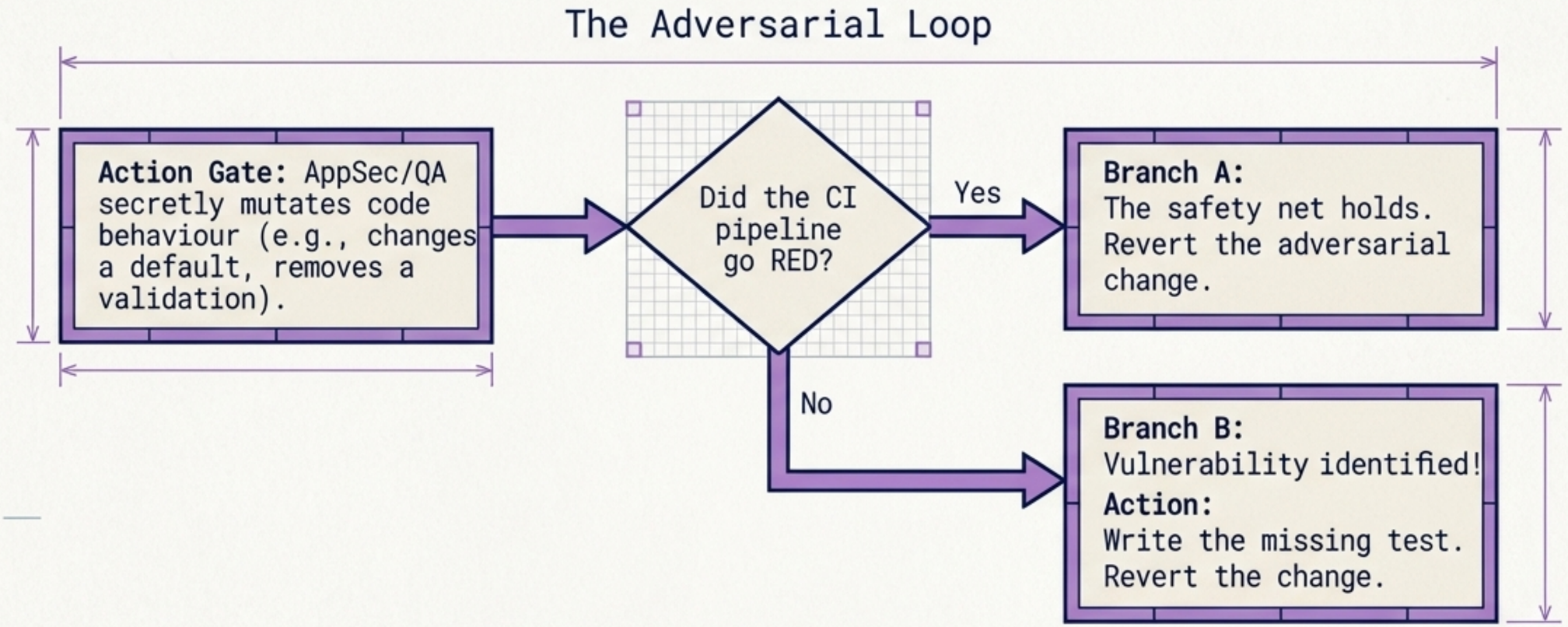


After



Objective:	Improve code structure without altering behaviour.
Actions:	Relocate code, abstract layers, eliminate duplication, fix naming, and separate concerns.
Mechanics:	Commit after every micro-step. If tests pass, the refactoring is safe. If a test breaks, fix it immediately.

Stress-Testing the Net: Adversarial Testing



The Goal: Reach a state where any behaviour-altering code modification triggers at least one test failure.

Phase 4: Next Wave (The Explorer Returns)

The Context

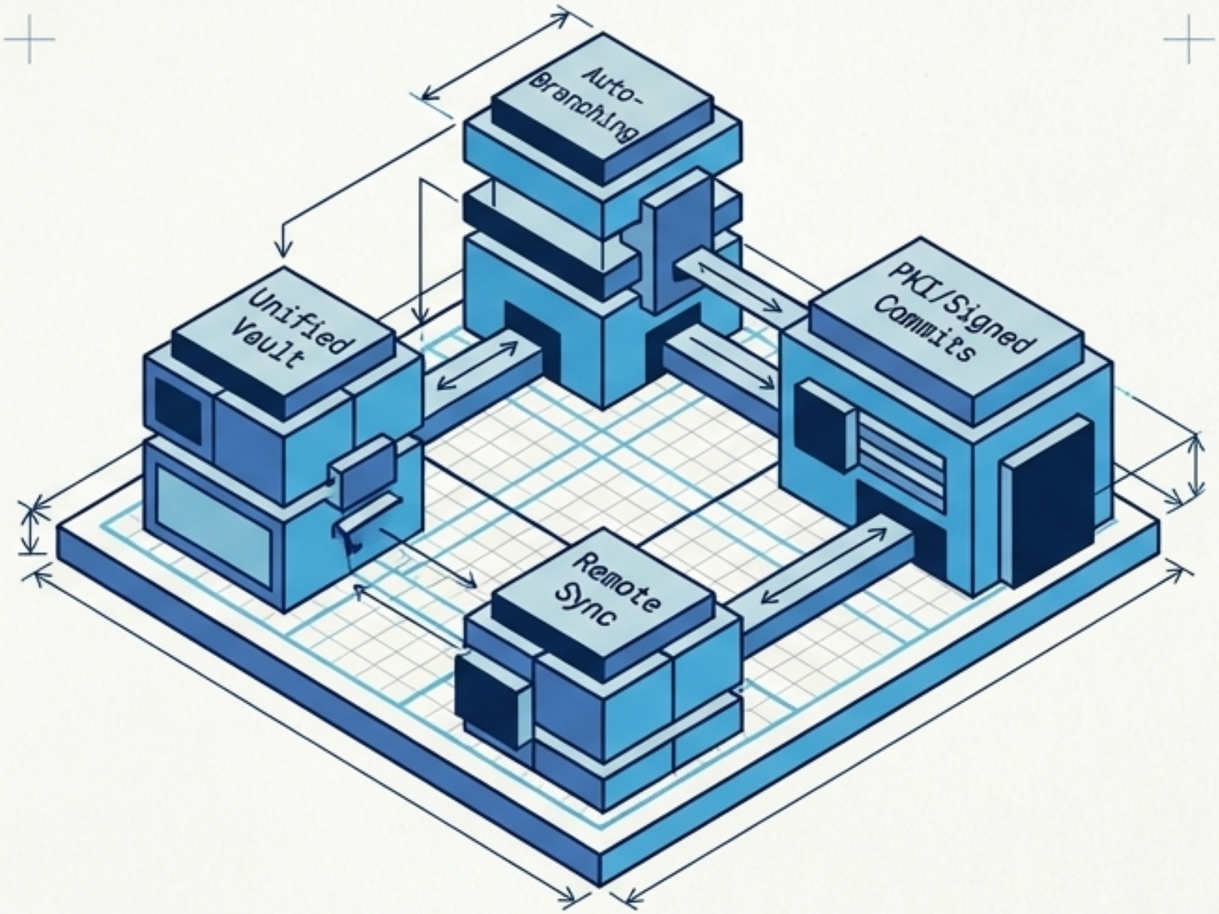
The codebase is now clean, tested, and structurally sound. The Explorer team returns to land the heavy features.

The Incoming Payload

- Unified vault structure (server-side and bare vault merged)
- Branch model (auto-branch on clone, PKI per branch)
- Signed commits & Remotes

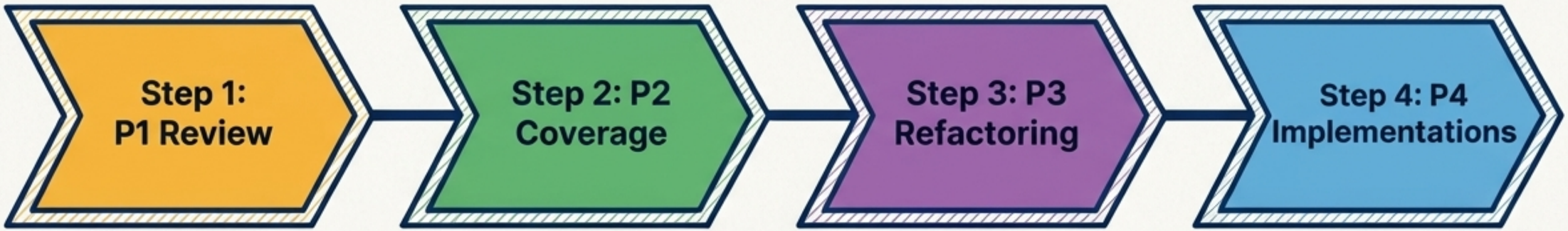
Paradigm Shift

Breaking changes are now expected and welcome. Every transition from version N to N+1 is fully visible and caught in the test diffs.



First Application: The CLI Project

Triggered by the Vault Mega-Memo identifying massive incoming architectural changes.



Execute P1 Architectural Review of the current mature CLI codebase.

Build P2 Coverage for all vault ops, CLI commands, and complete workflows.

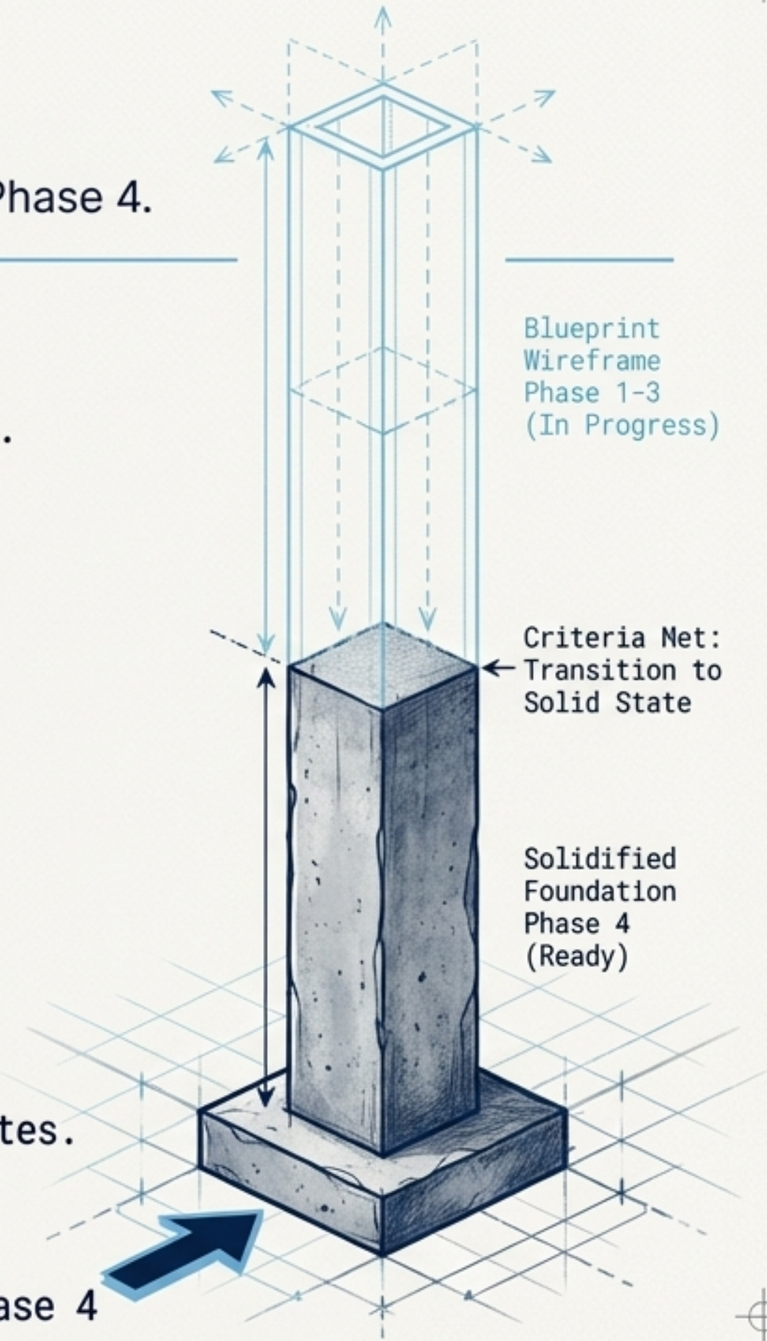
Perform P3 Refactoring to create clean abstractions for the remote/branch/merge additions.

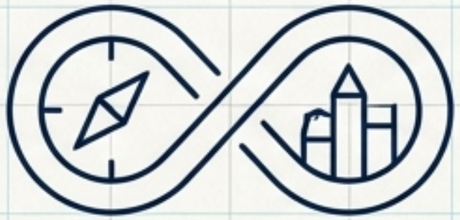
Land P4 Implementations: Branch model, unified vault, and remotes.

Acceptance Criteria

Only when these 9 criteria are met do we advance to Phase 4.

- [x] 1. Phase 1 architectural review completed (findings document produced).
- [x] 2. Test infrastructure implemented: local send server bootable in CI.
- [x] 3. Zero mocks or patches exist in the test suite.
- [x] 4. Code coverage reported automatically on every CI run.
- [x] 5. All known bugs captured securely as passing tests.
- [x] 6. All known vulnerabilities captured as passing tests.
- [x] 7. Adversarial testing completed: all behaviour modifications caught.
- [x] 8. Phase 3 refactoring completed with zero test regressions.
- [x] 9. CLI codebase verified ready for branch model, unified vault, and remotes.





The Explorer builds.
The Villager improves.